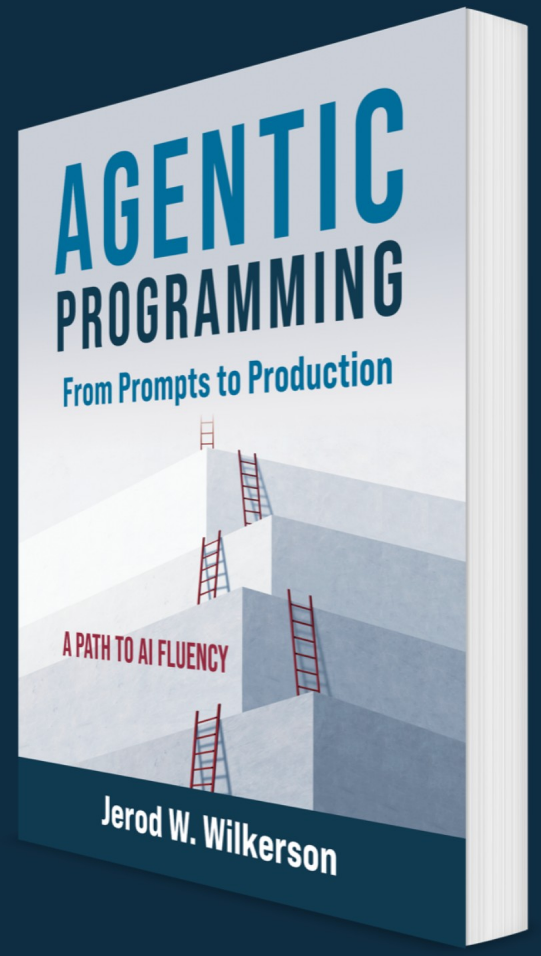
AGENTIC PROGRAMMING

CLIMBING THE AI FLUENCY LADDER

From AI-assisted coding to agentic programming

Jerod W. Wilkerson



The biggest thing AI changes about software development isn't how fast you write code.

IT'S HOW MUCH OF THE WORK YOU CAN HAND OFF.

THE MEASURE

THE QUESTION CHANGES

OLD QUESTION

How fast can I code?



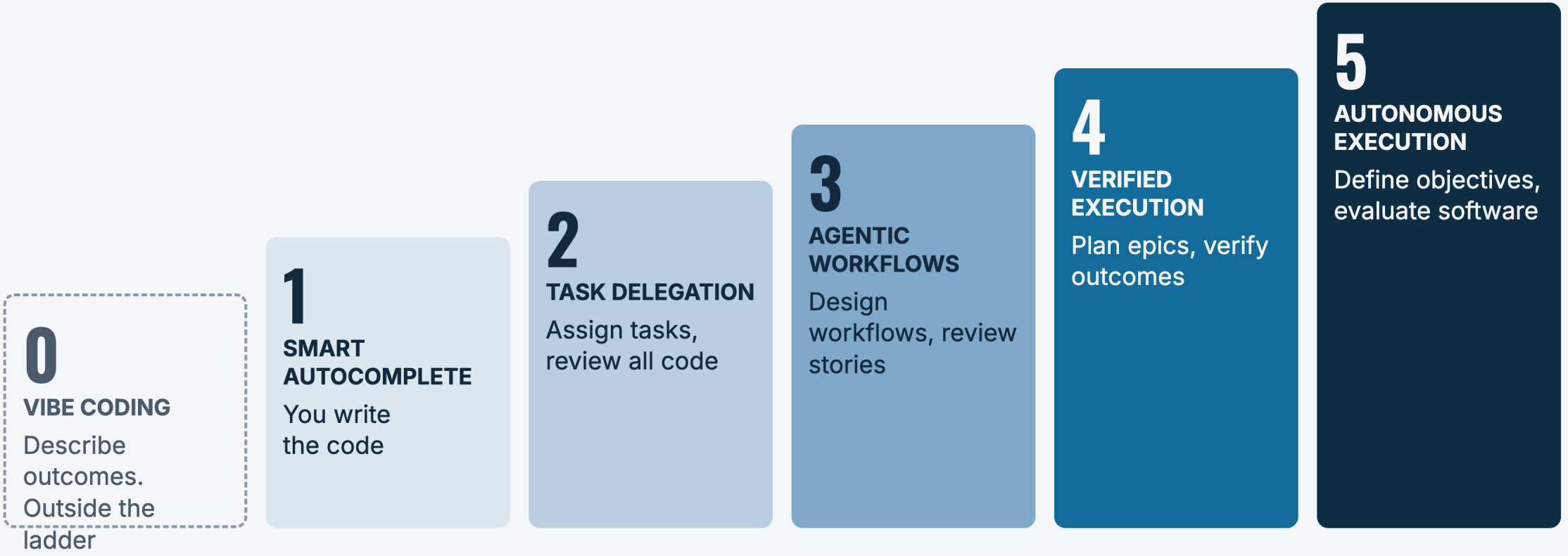
BETTER QUESTION

How much can I reliably delegate?

While keeping quality and control.

THE AI FLUENCY LADDER

EACH RUNG HANDS OFF MORE OF THE WORK



Framework by Dan Shapiro, popularized by Nate B. Jones, adapted in *Agentic Programming*.

WHY THE LEVELS EXIST

EACH LEVEL MOVES THE BOTTLENECK

You don’t climb because it’s fashionable. You climb because you hit the wall.

LEVEL 1	Your own implementation effort	Level 2 delegates tasks
LEVEL 2	Coordinating all that delegated work	Level 3 adds workflows
LEVEL 3	Reviewing every implementation	Level 4 verifies with evidence
LEVEL 4	Decisions you make while work runs	Level 5 runs without you
LEVEL 5	Defining objectives and judging results	**Still yours**

WHERE DEVELOPERS ARE

MOST DEVELOPERS ARE AT LEVEL TWO

0	1	2	3	4	5
GROWING RAPIDLY	COMMON	MAJORITY	EMERGING	RARE	EXPERIMENTAL

20–50%

The productivity gains developers report at level two. Real progress, but it tops out.

WHY PEOPLE STOP

WHY LEVEL TWO IS STICKY

01

It feels familiar

You still write code, with help. Nothing about your day has to change.

02

It’s the last level where you write production code

Above it, agents implement and your job becomes definition and verification.

03

The obstacle is mindset, not tools

For many developers, writing code is who they are.

THE CRAFT DOESN’T DISAPPEAR WHEN YOU CLIMB. IT MOVES.

HOW MODELS FAIL

CONFIDENCE IS A POOR PROXY FOR CORRECTNESS

Correct and incorrect answers arrive with the same fluency. You establish correctness from evidence, not tone.

EIGHT WAYS MODELS FAIL

01

Hallucination

Invents facts and states them with certainty

02

Silent failure

Nothing looks wrong. Something is missing.

03

Partial completion

“Mostly complete” looks like complete

04

Instruction drift

Rules you stated once stop applying

05

Overfitting to prompts

Follows the prompt, misses the outcome

06

Text parsing

Text is a fragile way to coordinate work

07

Sycophancy

Bends toward what you want to hear

08

Rationalization

Wrong answers with convincing reasoning

SILENT FAILURE

PROMPT SUCCESS IS NOT PROJECT SUCCESS

WHAT THE AGENT REPORTS

```
status: complete
tests: 41 passed, 0 failed
summary: "Endpoint implemented as specified."
```

WHAT THE STORY REQUIRED

- ✓ Create the endpoint
- ✓ Validate the input
- ✓ Store the record
- ⚠ **Rate limiting: missing, never built**

Illustration. The task succeeded; the requirement was never met.

SYCOPHANCY

AGREEMENT IS NOT EVIDENCE

FRAMED AS YOURS

Here’s the caching design I came up with. What do you think?

This is a sound approach. A few minor suggestions on key naming and expiration settings.

SAME DESIGN, FRAMED NEUTRALLY

A team is considering this caching design. What are its weaknesses?

Invalidation breaks under concurrent writes. The memory footprint may not fit your deployment. Consider an alternative.

Illustration from Chapter 4. The design didn’t change. Only who appeared to own it.

SYSTEMS > PROMPTS

Reliable outcomes come from the process around a prompt, not from perfecting the prompt itself.

The process around the prompt has a name: a harness.

THE HARNESS

THE SYSTEM THAT RUNS YOUR AI WORKFLOWS



Verification

Proves the work is done, with evidence instead of the agent's word for it



Retries

Sends failed work back, along with the reason it failed



Observability

Records every step, so you can see what happened and why

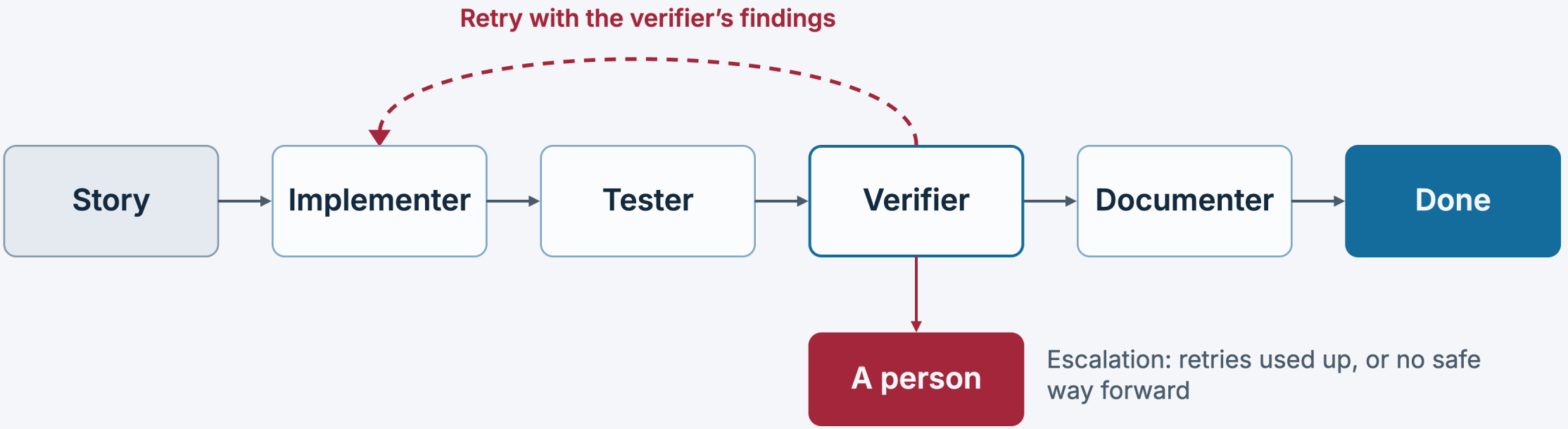


Escalation

Stops and hands the problem to a person when it can't proceed safely

LEVEL THREE

A STORY WORKFLOW



Coordinator: plain code that sequences stages, keeps state, runs the tests, and applies retry rules

DESIGN PRINCIPLE

DETERMINISTIC WHERE POSSIBLE, PROBABILISTIC WHERE NECESSARY

CODE: SAME INPUT, SAME RESULT

- Running the tests
- Linters and type checks
- Sequencing and state
- Retry rules

MODELS: JUDGMENT REQUIRED

- Implementing the story
- Writing the tests
- Judging the evidence
- Planning the work

A REAL RUN

ONE STORY, START TO VERIFIED

events.log — story-001

```
13:27:34 workflow started
13:29:48 implementer completed
13:31:06 tester completed
13:32:32 verification passed
13:33:17 documenter completed
13:33:17 story completed
```

5 min 43 s

Automated run, from start to a verified, documented story

The final, passing attempt. Planning and two earlier escalated attempts aren't included. From the harness built in Appendix A; dates trimmed.

THE FIRST ATTEMPTS

THE RUN THAT REFUSED TO PASS

1. Headless agents had no permission to run the tests.
2. The tester honestly recorded **tests_run: 0**.
3. The verifier refused to pass a story with no runtime evidence, and escalated.
4. **The code was fine.** Its 23 new tests passed when I ran them by hand.

VERIFIER, FIRST ESCALATION

```
"retry_recommended": false
"… operator action is required"
```

VERIFIER, SECOND ESCALATION

```
"This is an environment/harness-wiring
failure, not an implementation defect"
```

PRINCIPLE

FIX THE HARNESS, NOT THE CODE

- ✓ I didn’t touch the generated code.
- ✓ I didn’t weaken the verifier.
- ✓ I fixed how the harness granted permissions.

THE NUMBERS

ONE REAL PROJECT: HONOR CHECKER

~10×

labor leverage

My time: **at most 1 person-month**



Estimated work delivered: **9–12 person-months**



Dashed: the range of the estimate. Bars drawn to scale.

Under 10 hours a week for about 4 months, at levels three and four.

THE NUMBERS

THREE WAYS TO MEASURE IT

~5×

CALENDAR

18–24 months of estimated work delivered in about 4

~10×

LABOR

At most 1 person-month of my time against 9–12 estimated

~6×

OBSERVED PACE

Same developer, same system, before and after the harness. No estimate involved.

THE CAVEATS

WHAT DIDN'T SPEED UP

The architecture

Designed by me, the slow way

The specifications

What the epics ran against, written by hand

The harness itself

Real time to build, then to evolve from level three to four

One project, not a study. The baseline estimate is mine.

THE LEVERAGE SHOWS UP ONLY AFTER THE DEFINITION WORK IS DONE WELL ENOUGH TO DELEGATE AGAINST.

WHAT CHANGES FOR YOU

YOUR ROLE CHANGES AS YOU CLIMB

LEVELS 1–2

BUILDER

Writing and integrating code

LEVEL 3

DESIGNER

Defining stories and the verification that proves them

LEVEL 4

ORCHESTRATOR

Defining epics, reviewing outcomes, running work in parallel

LEVEL 5

VISIONARY

Defining products and objectives, judging what comes back

The roles add up. An Orchestrator still does Designer and Builder work.

WHAT CHANGES FOR YOU

SKILLS THAT GAIN AND LOSE VALUE

GAINING VALUE

- Judgment
- Specification
- Verification design
- Architecture

LOSING VALUE

- Implementation speed
- Syntax fluency
- Framework trivia

Understanding implementation keeps its value. You can’t specify what you don’t understand.

MONDAY MORNING

WHAT TO DO NEXT

1

Find your level honestly

Most developers are at level one or two. Honesty is cheap; self-flattery is expensive.

2

Climb on a real project

Practice projects don’t push back. Real acceptance criteria are where the skills form.

3

Build the next rung, not the top one

From level two, that’s a level-three harness that runs stories.

4

Write the acceptance criteria first

Even when you implement by hand. Design the verification for everything you delegate.

THE LADDER IS NOT A PLACE YOU LIVE

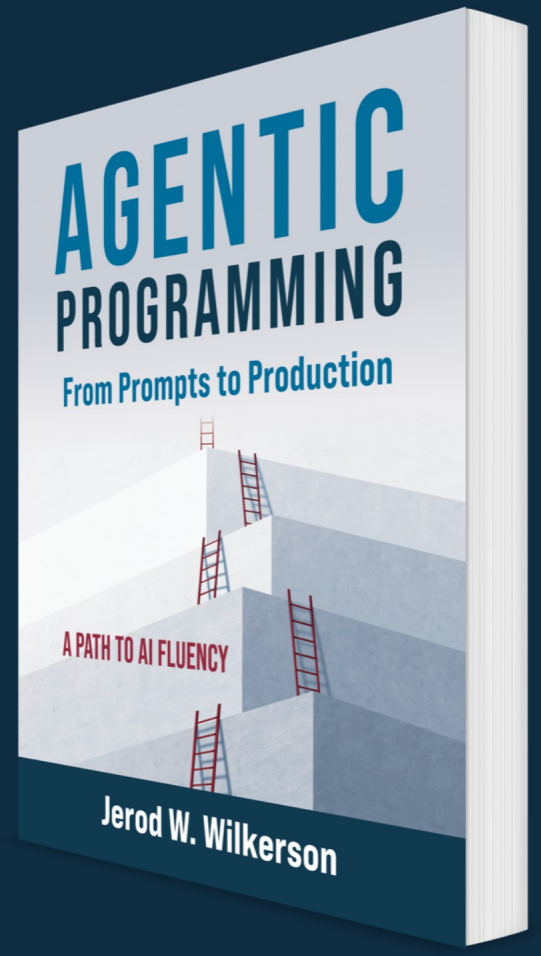
Pick a level per task. Climbing adds options without taking the old ones away.

Slides and resources

agenticprogrammingbook.com/talks

Code: github.com/jerodw/level-five

linkedin.com/in/jerodwilkerson



© 2026 Jerod W. Wilkerson. All rights reserved.